

Amazon EventBridge

Andrio Epping, Gabriel Silveira Marques e Bruna Letícia Schneiders

31/08/2026

Resumo

Este relatório analisa o Amazon EventBridge como solução de middleware orientado a eventos, situando sua arquitetura dentro da evolução histórica dos sistemas distribuídos, que partiram de integrações síncronas fortemente acopladas e migraram para modelos assíncronos baseados em mensagens e eventos. São descritos os mecanismos internos do EventBridge, entre eles o barramento de eventos, o casamento de padrões por conteúdo, o registro de schemas e as políticas de retentativa com backoff exponencial e filas de mensagens mortas. Na sequência, o trabalho compara o estilo arquitetural REST com a Arquitetura Orientada a Eventos (EDA) adotada pelo EventBridge, usando métricas de modularidade e complexidade estrutural extraídas de estudos empíricos, e contrasta o desempenho do EventBridge com o de outras plataformas de middleware corporativo, como Apache Kafka, RabbitMQ e Google Cloud Pub/Sub, considerando latência, vazão e custo operacional.

Sumário

1	Introdução	3
2	O Amazon EventBridge como Middleware	4
2.1	Por que sistemas distribuídos precisam de uma camada intermediária	4
2.2	Da comunicação síncrona ao roteamento por eventos	5
2.3	Arquitetura interna do EventBridge	6
2.4	Desacoplamento na prática do EventBridge	7

2.5	Resiliência e tratamento de falhas	8
3	Comparações	9
3.1	EventBridge como EDA frente ao estilo REST	9
3.2	EventBridge frente a outras plataformas de middleware	11
4	Conclusão	13

1 Introdução

Um sistema distribuído, segundo a definição clássica de Tanenbaum e Van Steen [1], é um conjunto de computadores independentes que se apresenta ao usuário como uma máquina única. O hardware é formado por nós autônomos, cada um com seu próprio processamento e memória, e o software precisa esconder essa fragmentação, dando a impressão de um sistema coeso. Manter essa impressão não é fácil porque exige resolver problemas de comunicação, consistência e tolerância a falhas que simplesmente não existem quando uma aplicação roda sozinha em uma única máquina.

A partir dos anos 2000, e de forma mais acentuada depois da popularização da computação em nuvem, as empresas passaram a fragmentar suas aplicações em serviços menores. Coulouris et al. [2] tratam essa fragmentação como resposta natural ao crescimento das demandas computacionais, mas ela também resolveu gargalos concretos de desempenho, disponibilidade e manutenção que os sistemas monolíticos já não conseguiam absorver.

As primeiras gerações de software distribuído usavam comunicação síncrona como base. O RPC (Remote Procedure Call) e, depois, as requisições HTTP no estilo REST dominaram a forma como os serviços se comunicavam. Isso funcionava bem enquanto o sistema tinha poucos componentes e a latência da rede era aceitável. O problema apareceu quando a complexidade aumentou, por exemplo: numa chamada síncrona, o serviço que faz a chamada precisa esperar a resposta antes de continuar. Quando o serviço do outro lado fica lento ou sai do ar, o problema não fica contido ali, ele se espalha por toda a cadeia de dependências. As threads ficam presas esperando, os recursos se esgotam, e uma falha pequena vira uma indisponibilidade geral. Esse fenômeno é conhecido na área como "monólito distribuído que é uma arquitetura que tem toda a complexidade operacional dos microsserviços, mas mantém o mesmo acoplamento rígido do monólito que deveria substituir.

Foi para lidar com esse tipo de falha em cascata que os sistemas modernos passaram a adotar comunicação indireta e assíncrona, em que um serviço publica uma mensagem ou evento e segue seu processamento sem esperar resposta imediata. Esse

relatório tem como objeto de estudo uma implementação específica desse modelo, o Amazon EventBridge, um barramento de eventos gerenciado pela AWS que permite que aplicações publiquem, filtrem e roteiem eventos entre si sem conhecer diretamente quem produz ou quem consome cada informação.

O EventBridge nasceu já como uma solução serverless, pensada para integrar serviços dentro e fora do ecossistema AWS sem exigir que a equipe de desenvolvimento gerencie servidores, partições ou filas manualmente. Isso o torna um bom contraponto às plataformas de middleware autogerenciadas, como Kafka e RabbitMQ, que aparecem ao longo deste relatório como referência comparativa.

O trabalho está organizado em três partes. A primeira apresenta os fundamentos de middleware necessários para entender o EventBridge e descreve sua arquitetura interna em detalhe, cobrindo o barramento de eventos, as regras de roteamento por conteúdo, o registro de schemas e os mecanismos de resiliência embutidos na plataforma. A segunda compara, com base em dados empíricos, a arquitetura orientada a eventos que o EventBridge representa com o estilo REST, e depois posiciona o EventBridge frente a outras tecnologias de middleware em termos de latência, vazão e custo de operação. A terceira reúne as conclusões sobre em que cenários o EventBridge é a escolha mais adequada e em que cenários outras plataformas cumprem melhor o papel.

2 O Amazon EventBridge como Middleware

2.1 Por que sistemas distribuídos precisam de uma camada intermediária

Middleware, em termos estruturais, é uma camada de software posicionada entre o sistema operacional e as aplicações distribuídas que rodam nos diversos nós de uma rede. Sua função é abstrair as diferenças naturais desses ambientes, seja diferença de plataforma, de linguagem de programação ou de protocolo de comunicação. Sem essa camada, cada aplicação teria que lidar diretamente com essas incompatibilidades, multiplicando o esforço de desenvolvimento e deixando o sistema frágil a qualquer

mudança na infraestrutura.

Podemos imaginar um cenário onde uma empresa pode ter um sistema de vendas em Java, um módulo de logística em Python, um serviço de pagamentos que expõe API REST e um sistema legado de contabilidade que ainda usa protocolos proprietários. Fazer esses componentes conversarem sem uma camada de abstração exigiria que cada equipe conhecesse os detalhes internos das outras, o tipo de acoplamento que a engenharia de software tenta evitar desde sempre. É exatamente esse problema que o EventBridge se propõe a resolver, oferecendo um ponto único de publicação e roteamento para eventos originados em qualquer parte do sistema.

2.2 Da comunicação síncrona ao roteamento por eventos

As primeiras implementações de middleware, como CORBA (Common Object Request Broker Architecture) e as chamadas RPC tradicionais, eram fortemente acopladas. A premissa era permitir que um programa em uma máquina chamasse métodos em outra máquina como se fossem chamadas locais. Funcionava, mas com restrições sérias, já que a comunicação era síncrona, o acoplamento entre emissor e receptor era forte, e a escalabilidade ficava limitada pela capacidade do servidor que hospedava o objeto remoto.

Com o tempo, essas limitações levaram a dois modelos de comunicação assíncrona que hoje dominam o mercado.

- **MOM (Message-Oriented Middleware).** Gerencia comunicação por meio do transporte e roteamento de mensagens formatadas, tipicamente organizadas em filas ponto a ponto. Um produtor deposita a mensagem em uma fila física, e um consumidor a retira para processar. A fila funciona como amortecedor temporal, retendo a mensagem até que o consumidor esteja disponível. O Amazon SQS é o exemplo mais direto desse paradigma na nuvem, com filas gerenciadas e retenção de até 14 dias.
- **EOM (Event-Oriented Middleware).** Opera sob lógica diferente, propagando mudanças de estado, os eventos, para múltiplos interessados ao mesmo tempo, geralmente no modelo publicador assinante. Um componente publica

um evento em um barramento ou tópico, e todos os assinantes registrados recebem a notificação de forma autônoma, sem que o publicador precise saber quantos consumidores existem. É nesse paradigma que o EventBridge se encaixa, junto de plataformas como o Google Cloud Pub/Sub.

Importante salientar que essa fronteira nem sempre é nítida na prática. O Apache Kafka [7], por exemplo, mistura características dos dois modelos, já que usa tópicos com múltiplos assinantes, mas armazena as mensagens em um log persistente que os consumidores leem por operações de pull. O EventBridge, em contraste, segue um modelo de push mais puro, empurrando o evento para os alvos configurados assim que uma regra de roteamento é satisfeita, sem exigir que o consumidor faça polling ativo.

2.3 Arquitetura interna do EventBridge

O núcleo do EventBridge é o event bus, um canal lógico para o qual eventos são publicados. Toda conta AWS já nasce com um bus padrão, que recebe automaticamente eventos gerados por diversos serviços da própria AWS, mas é possível criar barramentos customizados dedicados a aplicações específicas, e também barramentos de parceiros, usados por fornecedores externos que integram seus produtos SaaS diretamente na infraestrutura do cliente.

Cada evento que chega ao barramento é avaliado contra um conjunto de regras. Uma regra é composta por um padrão de evento, escrito em JSON, que especifica quais campos e valores do evento devem casar para que a regra dispare. Esse mecanismo é uma implementação prática do padrão Content-Based Router descrito por Hohpe e Woolf [3] nos Enterprise Integration Patterns. Em vez de cada consumidor receber todos os eventos e filtrar localmente os que lhe interessam, a filtragem acontece dentro do próprio barramento, o que evita desperdício de processamento nas pontas e reduz o tráfego desnecessário entre os serviços.

Quando uma regra casa com um evento, o EventBridge o encaminha para um ou mais alvos, que podem ser funções Lambda, filas SQS, tópicos SNS, máquinas de estado do Step Functions, endpoints HTTP de APIs externas, ou dezenas de

outros serviços AWS. Essa flexibilidade de destino é um dos motivos pelos quais o EventBridge costuma ser descrito como um hub de integração, e não apenas como uma fila de mensagens.

Outros dois recursos merecem destaque por endereçarem problemas que, em arquiteturas mais antigas, precisavam ser resolvidos manualmente em cada aplicação.

- **Schema Registry.** O EventBridge pode inferir e armazenar automaticamente o formato dos eventos que trafegam por um barramento, gerando um schema versionado que outras equipes podem consultar para saber exatamente quais campos esperar. Isso reduz o retrabalho de documentação manual de contratos entre times, um problema recorrente em integrações que crescem ao longo do tempo.
- **Archive e Replay.** Eventos publicados em um barramento podem ser arquivados automaticamente por um período configurável e reproduzidos posteriormente. Esse recurso é útil tanto para depuração de falhas, permitindo recriar exatamente a sequência de eventos que levou a um erro, quanto para popular novos consumidores com o histórico de eventos relevante ao entrarem no sistema.

Uma adição mais recente à plataforma, o EventBridge Pipes, simplifica a conexão ponto a ponto entre uma fonte de eventos e um destino, permitindo transformação e enriquecimento do payload no meio do caminho sem exigir uma função Lambda dedicada só para essa tarefa. Isso reduz a quantidade de código de cola que equipes precisavam escrever para adaptar o formato de um evento entre sistemas diferentes.

2.4 Desacoplamento na prática do EventBridge

A eficácia de um middleware pode ser medida pela capacidade de garantir desacoplamento entre os componentes que ele interliga, e no caso do EventBridge esse desacoplamento aparece em três frentes.

O primeiro é o desacoplamento espacial. Um serviço que publica um evento no barramento não precisa saber a identidade, o endereço ou sequer a quantidade de

consumidores interessados naquela informação. Toda a responsabilidade de resolver quem deve receber o evento fica com as regras configuradas no EventBridge. Isso significa, na prática, que um novo consumidor pode ser adicionado ao sistema criando apenas uma nova regra, sem tocar em uma linha sequer do serviço produtor.

O segundo é o desacoplamento temporal. Em uma chamada síncrona convencional, produtor e consumidor precisam estar ativos ao mesmo tempo. Formalmente, o estado de acoplamento $S_{coupling}$ entre um produtor P e um consumidor C exige

$$S_{coupling} = P(t) \cap C(t) \neq \emptyset \quad (1)$$

Com o EventBridge atuando como intermediador Q , a interação se divide em dois momentos independentes,

$$P(t_1) \rightarrow Q \quad \text{e} \quad Q \rightarrow C(t_2) \quad \text{onde} \quad t_1 \neq t_2 \quad (2)$$

Na prática, isso significa que uma manutenção programada em um consumidor no instante t_1 não afeta o serviço produtor. O EventBridge tenta entregar o evento ao alvo, e se o alvo estiver indisponível, ele aplica sua política de retentativa até que o consumidor volte a responder.

O terceiro é o desacoplamento de sincronização. A publicação de um evento no EventBridge é uma operação não bloqueante do ponto de vista de quem publica. O produtor envia o evento e retoma imediatamente suas próprias operações, sem aguardar confirmação de que o consumidor final o processou. Do ponto de vista do código do produtor, essa chamada se comporta como uma operação local rápida, mesmo que a entrega efetiva ao alvo ocorra segundos depois.

2.5 Resiliência e tratamento de falhas

Quando a entrega de um evento a um alvo falha, o EventBridge não tenta reenviá-lo de imediato. A plataforma espaça as tentativas de forma crescente, começando por um segundo, depois dois, depois quatro, e assim sucessivamente, um mecanismo conhecido como backoff exponencial, que evita sobrecarregar um consumidor que já está enfrentando problemas.

A esse mecanismo o EventBridge adiciona uma variação aleatória nos intervalos de espera, o chamado jitter. Sem esse componente aleatório, múltiplos produtores que falhassem simultaneamente sincronizariam suas retentativas, gerando picos de carga periódicos justamente no momento em que o consumidor tenta se recuperar. O jitter distribui essas tentativas de forma mais homogênea ao longo do tempo.

Eventos que esgotam todas as tentativas de retentativa sem sucesso são encaminados para uma fila de mensagens mortas, a Dead Letter Queue. Essa fila funciona como repositório de diagnóstico, permitindo que a equipe de operações inspecione os eventos não entregues, identifique a causa raiz da falha e, se for o caso, reprocessse manualmente aquele lote depois de corrigido o problema no consumidor.

Essas três funcionalidades, geridas inteiramente pelo EventBridge, poupam as aplicações de negócio de uma quantidade considerável de lógica de infraestrutura. Em um cenário sem esse suporte nativo, cada serviço precisaria implementar seus próprios mecanismos de retentativa e registro de falhas, o que multiplicaria o esforço de desenvolvimento e ainda correria o risco de gerar inconsistências entre os diferentes times responsáveis por cada serviço.

3 Comparações

3.1 EventBridge como EDA frente ao estilo REST

A decisão de adotar um barramento de eventos como o EventBridge em vez de integrações REST tradicionais não deveria se apoiar apenas em preferência pessoal da equipe. Lazzari e Farias [4] conduziram um estudo empírico em que implementaram um mesmo sistema de gestão de pedidos sob as duas abordagens e submeteram ambas a cinco cenários evolutivos, medindo métricas estruturais a cada ciclo. Embora o estudo não use o EventBridge especificamente, seus resultados se aplicam diretamente a qualquer implementação EDA baseada em barramento de eventos, o EventBridge incluso.

A primeira métrica relevante é a separação de interesses. Na implementação orientada a eventos, a introdução de novas funcionalidades afetou em média 40,74% menos módulos do que na implementação REST, medido pelo CDC (Concern Diffu-

sion over Components). As transições de interesse em linhas de código, o CDLOC, foram 48,71% menores na abordagem EDA. Esses números sustentam algo que faz sentido intuitivamente ao pensar em como o EventBridge funciona, já que quando os serviços se comunicam publicando e assinando eventos em vez de se chamarem diretamente, uma mudança em um serviço tende a ficar mais contida dentro dele mesmo.

Por outro lado, o mesmo estudo mostra um resultado que parece contraditório à primeira vista. As métricas de dependência direta cresceram na direção oposta, com a dependência interna subindo 575% e a externa 237,5% no cenário EDA. A explicação é técnica e se aplica bem ao EventBridge, porque mesmo que os módulos de negócio não precisem mudar simultaneamente, toda a infraestrutura de eventos depende de definições estruturais compartilhadas, os schemas dos eventos, as interfaces de serialização, os contratos de cada regra de roteamento. É justamente para reduzir esse tipo de atrito que o Schema Registry do EventBridge existe, centralizando essas definições em um único lugar consultável por todas as equipes envolvidas.

A coesão relacional de interfaces também foi 26,48% mais favorável na implementação EDA, sugerindo que cada classe tende a ficar mais focada quando o sistema adota eventos. O contraponto aparece na complexidade estrutural, que disparou 572,09% na abordagem EDA, e no volume de código, com um crescimento de 76,52% em linhas lógicas e 70,37% no número de atributos instanciados. Essa escalada reflete algo que qualquer equipe que já implementou uma integração com o EventBridge reconhece, já que para cada tipo de evento é preciso definir schemas de serialização, escrever os padrões de casamento das regras, configurar os alvos e, com frequência, manter classes utilitárias de transformação entre formatos internos e externos. Cada peça isolada é simples, mas o volume acumulado ao longo de um sistema real não é desprezível.

Esses números indicam que a escolha entre REST e um barramento como o EventBridge não é uma questão de superioridade absoluta, e sim de perfil de projeto. Sistemas que evoluem com frequência e que precisam de resiliência a falhas tendem a se beneficiar do modelo orientado a eventos, enquanto sistemas mais estáveis, com fluxos bem definidos e equipes pequenas, podem achar no REST uma solução mais

direta de manter.

3.2 EventBridge frente a outras plataformas de middleware

Escolher entre REST e EDA resolve só metade do problema. É preciso também decidir qual tecnologia concreta vai implementar esse barramento de eventos, e essa escolha envolve trocas que nem sempre são óbvias. A Tabela 1 resume o modelo de comunicação e a arquitetura base de sete plataformas amplamente usadas no mercado, incluindo o EventBridge.

Tabela 1: Modelos de Comunicação e Arquitetura de Middlewares

Tecnologia	Modelo de Comunicação	Arquitetura Base	Escalabilidade e Confiabilidade
EventBridge	Barramento de Eventos (Push-to-Push)	Serverless multi-tenant na nuvem AWS	Elástica sob demanda, com retentativas e DLQ
Apache Kafka	Log Distribuído (Pull)	Cluster com armazenamento em disco	Massiva horizontal, alta persistência
Apache Pulsar	Multi-tenant Separado	Brokers independentes do storage	Horizontal com geo-replicação nativa
RabbitMQ	Broker de Filas AMQP (Push-to-Pull)	Servidores gerenciando filas em disco/RAM	Escala vertical, garantias de entrega (acks)
Amazon SQS	Message Queue (Pull)	Serverless de fila ponto a ponto	Praticamente ilimitada, até 14 dias de retenção
GCP Pub/Sub	Tópicos Pub/Sub Global	Serverless integrado (Google Cloud)	Global automática, persistência distribuída
Redis Streams	Memória otimizada	Cluster in-memory	Latência ultrabaixa com retenção efêmera

O EventBridge se diferencia das demais plataformas justamente por abstrair completamente a infraestrutura subjacente. Não existe cluster para provisionar, partição para dimensionar ou disco para monitorar. A AWS gerencia tudo em um ambiente multi-inquilino compartilhado, o que reduz o esforço operacional da equipe, mas também limita o controle fino sobre parâmetros de desempenho e introduz alguma variabilidade na latência, algo que fica claro quando se olha os números de benchmarking.

A Tabela 2 traz os dados empíricos de desempenho obtidos nos estudos de Gopisetty [5] e Arafat, Tasmin e Poudel [6], que avaliaram essas tecnologias sob condições controladas de carga.

Tabela 2: Benchmarking Operacional e Parâmetros Físicos Sob Carga de Teste

Framework	Vazão Máxima (msg/seg)	Latência p95 (ms)	Complexidade (FTE)	Limitação Chave
Apache Kafka	~ 1.200.000	18	2,3	Rebalanceamento frio, particionamento estático
Apache Pulsar	~ 950.000	22	1,8	Complexidade arquitetural (BookKeeper)
NATS JetStream	~ 800.000	15	1,2	Limitações no tamanho da mensagem e memória
Redis Streams	~ 650.000	8	0,8	Retenção ligada à capacidade de memória RAM
RabbitMQ	~ 450.000	32	1,5	Teto de throughput, alto overhead de rotas
GCP Pub/Sub	~ 300.000	78	0,4	Oscilações de latência regional
EventBridge	~ 200.000	85	0,3	Piso de latência rígido, controle de taxa

À primeira vista, o EventBridge parece perder feio nessa comparação, ocupando a última posição tanto em vazão máxima quanto em latência p95. Mas olhar só para essas duas colunas é enganoso. A coluna de complexidade operacional, medida em FTE, equivalentes em tempo integral de engenheiro dedicados a manter a infraestrutura, inverte boa parte dessa leitura. O Apache Kafka, apesar de processar mais de um milhão de mensagens por segundo, demanda o equivalente a 2,3 engenheiros só para gerenciar partições, monitorar réplicas, ajustar configurações de rebalanceamento e aplicar patches de segurança. O EventBridge exige 0,3 FTE, uma fração pequena desse esforço, porque a AWS absorve toda essa carga operacional.

Isso posiciona o EventBridge de forma clara dentro do mapa de decisão. Para cenários que envolvem liquidações financeiras em tempo real, telemetria industrial de altíssima frequência ou qualquer aplicação que exija ordenamento transacional rigoroso, plataformas autogerenciadas como Kafka ou Pulsar seguem sendo praticamente obrigatórias, já que nenhuma solução serverless atual entrega throughput e latência equivalentes com log imutável e reprocessamento garantido. Nesses casos, o custo humano de operar Kafka se justifica pela criticidade da aplicação.

Já em integrações entre microsserviços internos, coreografia de processos de negócio, notificações entre domínios distintos ou conexão com dezenas de serviços SaaS de terceiros, o cenário muda de figura. Uma latência de 85 milissegundos é perfeitamente aceitável quando os consumidores não dependem de resposta em dezenas de milissegundos, e a economia operacional que o EventBridge proporciona costuma pesar mais do que o ganho bruto de desempenho de uma plataforma autogerenciada. Não existe disco para monitorar, não existe cluster para redimensionar, e a Dead Letter Queue já vem configurada por padrão.

O Redis Streams ocupa um nicho particular, com latência p95 de apenas 8 milissegundos e FTE de 0,8, combinando baixa latência com custo operacional moderado, mas sua limitação estrutural é a retenção atrelada à memória RAM disponível, o que o torna pouco adequado para reprocessamento de grandes volumes históricos, algo que o EventBridge resolve de forma nativa com seu recurso de archive e replay.

O que esses dados deixam claro é que não existe uma plataforma de middleware simultaneamente mais rápida, mais barata e mais simples de operar. A escolha da tecnologia é, por natureza, um exercício de troca entre desempenho bruto e custo operacional, e entre controle granular sobre a infraestrutura e conveniência gerenciada. O EventBridge se posiciona claramente do lado da conveniência gerenciada, e essa posição é uma escolha de projeto, não uma limitação acidental da plataforma.

4 Conclusão

A análise desenvolvida ao longo deste relatório permite chegar a algumas conclusões fundamentadas sobre o papel do Amazon EventBridge dentro do panorama atual de sistemas distribuídos.

A primeira é que o EventBridge representa bem a migração dos modelos síncronos para os assíncronos que caracteriza a engenharia de software das últimas duas décadas. Essa migração responde a limitações técnicas concretas, e o bloqueio de threads imposto por chamadas síncronas limita de forma determinística a capacidade de resposta horizontal de um sistema, algo que o modelo de push do EventBridge contorna ao permitir que produtores sigam operando sem esperar confirmação de entrega.

A segunda é que a arquitetura orientada a eventos que o EventBridge implementa traz ganhos reais de modularidade, reduzindo a difusão de interesses entre componentes, mas cobra esse benefício em complexidade estrutural adicional. Schemas, regras de casamento de padrões e configuração de alvos exigem disciplina para não se tornarem, na prática, tão opacos quanto o acoplamento que pretendiam eliminar. Recursos como o Schema Registry ajudam a mitigar esse custo, mas não o eliminam.

A terceira é que a posição do EventBridge no mercado de middleware é bem

definida. Ele não compete diretamente com Kafka ou Pulsar em cenários de altíssimo volume e baixíssima latência, e não deveria ser escolhido para esse tipo de carga. Sua força está em integrações corporativas, conexão nativa com o ecossistema AWS e serviços de terceiros, e na quase total eliminação do esforço de gestão de infraestrutura, o que fica evidente na comparação de FTE frente às plataformas autogerenciadas.

Não existe framework que resolva simultaneamente todos os problemas de latência, tolerância a falhas e modularidade. A escolha do EventBridge para um projeto específico deve resultar da ponderação entre o isolamento que a arquitetura orientada a eventos proporciona e os recursos, humanos e computacionais, efetivamente disponíveis para operar a solução escolhida. Para a grande maioria das integrações corporativas que não exigem latência de milissegundos nem throughput de milhões de mensagens por segundo, essa ponderação tende a favorecer o EventBridge.

Referências Bibliográficas

- [1] TANENBAUM, Andrew S.; VAN STEEN, Maarten. *Distributed Systems: Principles and Paradigms*. 1. ed. Nova Jérsei: Prentice Hall, 2002. ISBN 9780130888938.
- [2] COULOURIS, George; DOLLIMORE, Jean; KINDBERG, Tim; BLAIR, Gordon. *Distributed Systems: Concepts and Design*. 5. ed. Boston: Addison-Wesley, 2012. ISBN 9780132143011.
- [3] HOHPE, Gregor; WOOLF, Bobby. *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. 1. ed. Boston: Addison-Wesley, 2003. ISBN 9780321200686.
- [4] LAZZARI, Luan; FARIAS, Kleinner. Event-driven Architecture and REST Architectural Style: An Exploratory Study on Modularity. *Computación y Sistemas*, Cidade do México, v. 27, n. 3, 2023. Disponível em: https://www.scielo.org.mx/scielo.php?script=sci_arttext&pid=S1665-64232023000300338.
- [5] GOPISETTY, Satyanarayana. Exactly-Once, Always Auditable: Benchmarking the Latency, Throughput, and Evidential Integrity Trade-offs of AWS Serverless Orchestration versus Choreography for High-Frequency Payment Settlements. *International Journal of Computer Technology (IACSE-IJCT)*, v. 7, n. 1, p. 14-36, 2026. DOI: 10.5281/zenodo.20266481.
- [6] ARAFAT, Jahidul; TASMIN, Fariha; POUDEL, Sanjaya. Next-Generation Event-Driven Architectures: Performance, Scalability, and Intelligent Orchestration Across Messaging Frameworks. *arXiv preprint arXiv:2510.04404*, 2025. Disponível em: <https://arxiv.org/pdf/2510.04404>.
- [7] ALAASAM, A. B.; RADCHENKO, G.; TCHERNYKH, A. Stateful stream processing for digital twins: Microservice-based Kafka stream DSL. In: *International Multi-Conference on Engineering, Computer and Information Sciences (SIBIRCON)*. Novosibirsk: IEEE, 2019. p. 804-809.

- [8] AMAZON WEB SERVICES. *What Is Amazon EventBridge?*. AWS Documentation, 2019. Disponível em: <https://docs.aws.amazon.com/eventbridge/latest/userguide/eb-what-is.html>.